DB2HACK 2026 · ROUND 1 PROPOSAL · IBM DB2 + AGENTIC AI

# Hotel Ops Copilot

A team of AI agents that runs a hotel's daily operations on **IBM Db2**. The AI only **proposes**; Db2 decides what it may read, **dry-runs** every action, enforces the business rules, and can **undo** any change.

> "The database governs the AI, not the prompt."

**30/30**
evaluation questions correct, checked against ground truth computed in Db2

**24/24**
red-team attacks blocked, 22 of them by Db2 itself

**15+**
Db2 capabilities doing real work: vectors, temporal tables, SQL PL, RCAC, EXPLAIN…

**~4**
LLM calls per question on average (30-question evaluation); a full morning brief takes ~16

**Participant** Clark Fernandes (solo) · GitHub clarkf19

**Framework** LangGraph + LangChain

**Database** IBM Db2 12.1 Community Edition

**Status** working prototype, 106 automated tests

THE PROBLEM

Problem statement

# Every morning, a hotel duty manager must make revenue, guest and maintenance decisions from scattered data, and AI is too risky to hand the keys to

### 🧩 Fragmented insight

Occupancy lives in the property-management system, rates in a spreadsheet, and reviews across Google, TripAdvisor and OTAs. A rattling AC in room 307 is noticed only after **many** bad reviews, because nobody connects "aircon humming" and "AC noise" to one room.

### 📉 Slow, manual revenue decisions

Mid-size and independent hotels cannot afford revenue-management systems or analysts. Weak nights are discounted too late, sold-out conference nights are under-priced, and loyal VIPs arrive without anyone noticing it is their anniversary.

### ⚠️ AI is unsafe to deploy

"Chat with your database" tools can run any SQL, leak guest PII to a third-party model, follow instructions hidden in a review (prompt injection), and make changes nobody can audit or **undo**.

### Who feels it

- **Duty and general managers** of independent and mid-size hotels: the morning brief, approvals and escalations
- **Revenue managers**: pricing soft and peak nights inside policy
- **Engineering and front office**: fixing the right room first, upgrading the right guest

### The trust gap

No hotel will let an AI change prices or block rooms unless every change is **explainable, checked against business rules, approved by a human and reversible**. Prompts cannot guarantee that. **The database can.**

THE SOLUTION

Proposed solution

# A governed multi-agent copilot: the AI proposes, a human approves, and Db2 enforces and can undo

The manager talks in plain English. LangGraph agents read Db2, find problems and opportunities, and queue evidence-backed actions. Every safety guarantee lives **inside IBM Db2**.

**1 · Ask**
"Give me this morning's brief" or any question in plain English

**2 · Analyse**
Router plus 3 specialist agents query Db2 (SQL, vectors, time travel)

**3 · Propose**
Actions go through `SP_PROPOSE_ACTION`, which validates them and blocks bad ones

**4 · Dry run**
Db2 executes the action in a transaction, shows its impact, then rolls back

**5 · Approve**
A human manager decides; `SP_APPLY_ACTION` re-checks the guardrails

**6 · Undo**
Any change is rolled back with `FOR SYSTEM_TIME AS OF`

### Morning brief

One click: KPIs, guest issues and the actions to approve, composed from three agents' findings.

### Complaint radar

Db2 vector search sorts every review sentence into themes and finds *where* each one clusters.

### Dry run

See exactly what an approval would change, including oversold nights, before you approve.

### Time travel

Every rate and booking version is kept; there's a change feed and one-click rollback.

### Text-to-SQL analyst

Occupancy, ADR, RevPAR and pick-up from Db2 views, grounded in catalog comments.

### Policy-grounded

The hotel handbook lives in Db2 via IBM `langchain-db2`; proposals cite policy IDs.

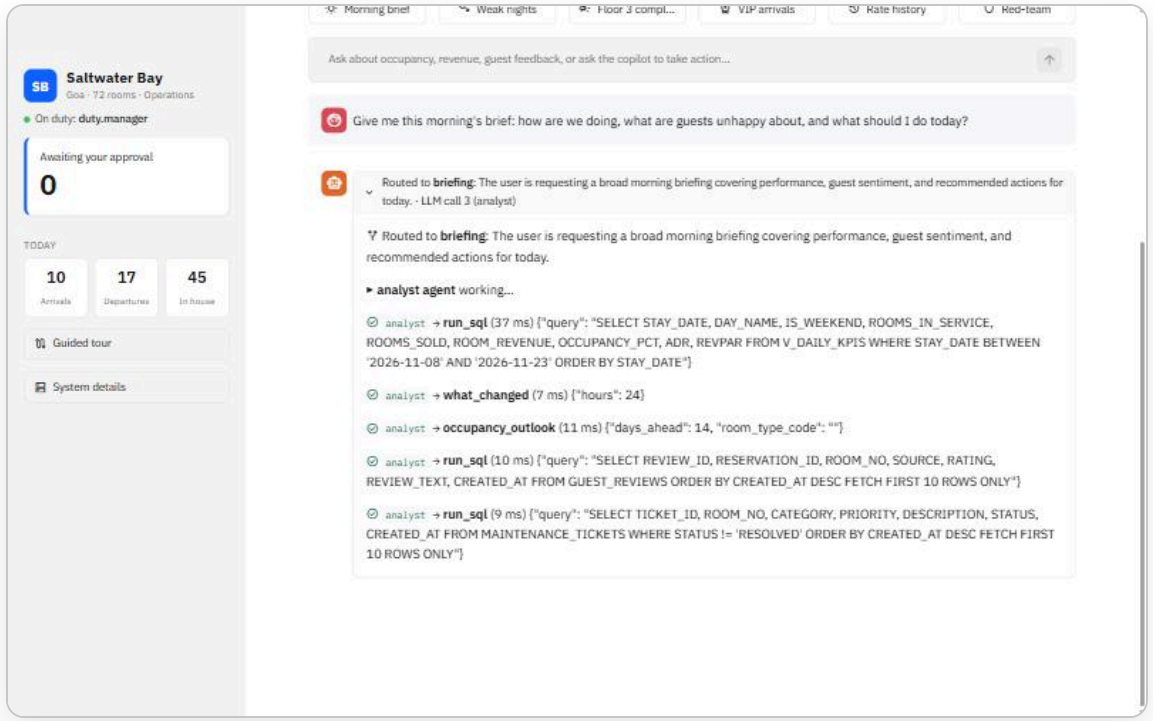
### Privacy by Db2

Guest e-mail and phone are masked for the AI; opted-out guests are invisible to it (DPDP Act 2023).

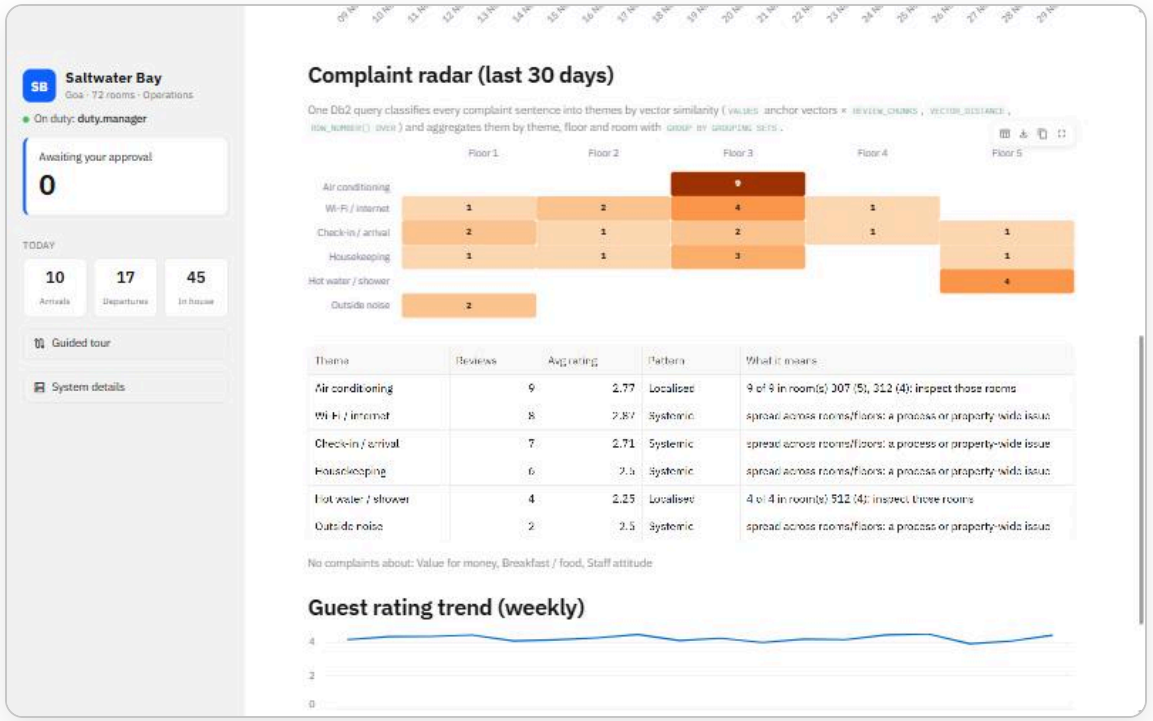
### Earned autonomy

Managers grant the AI low-risk autonomy (e.g. ±5% price nudges). Db2 enforces the limits, and every change can be undone. 24/24 attacks are blocked.

# From question to governed action in under a minute




**Live multi-agent trace.** The router picks the briefing route, then the analyst calls Db2 tools (SQL, change feed, outlook) with timings shown live. Every step is stored in Db2.




**Complaint radar.** One Db2 query classifies every complaint sentence. AC (rooms 307/312) and hot water (512) are *localised* faults; Wi-Fi and check-in are *systemic*.

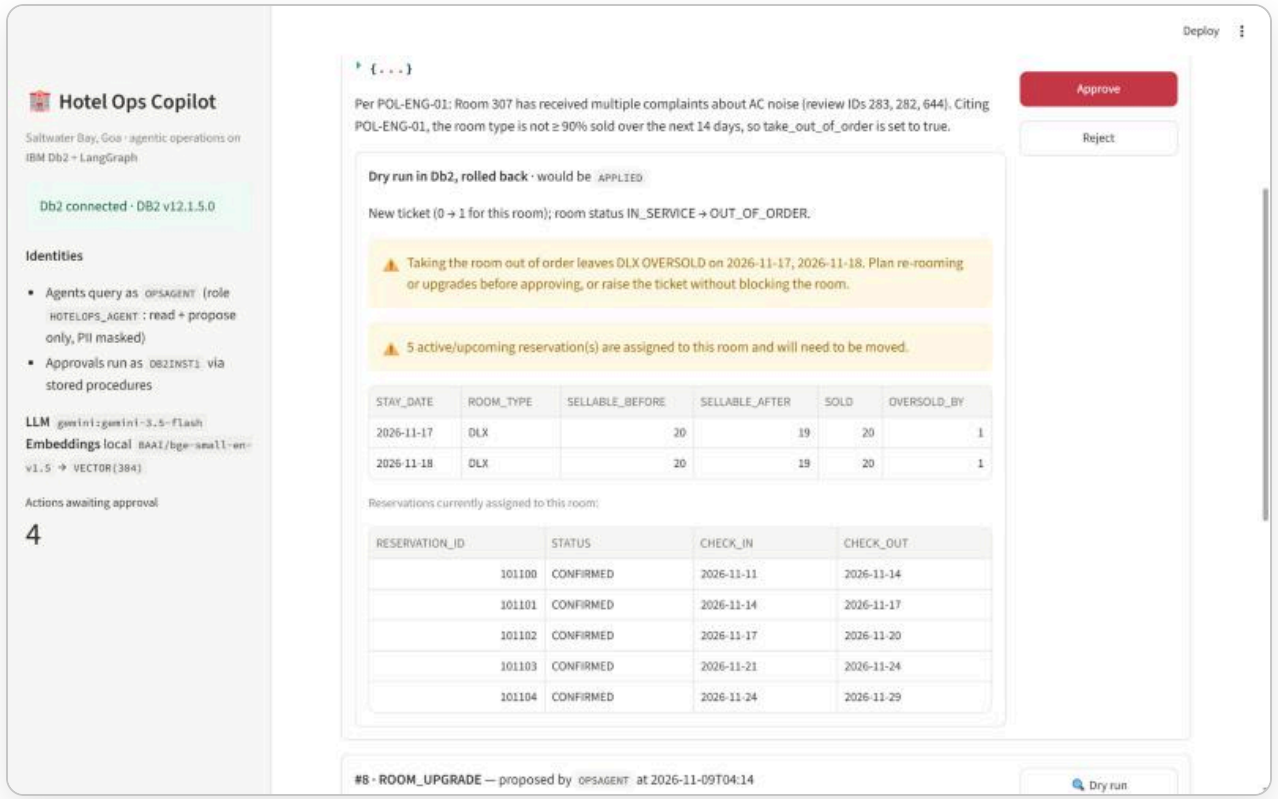
**~16**
LLM calls for a full morning brief across four agents (router, 3 specialists, composer)

**12**
Db2 tool calls in that brief: SQL, vectors, time travel and change feed

**6**
evidence-backed actions queued: two repairs, a VIP upgrade, three rate changes

**100%**
of steps stored in Db2: replayable, auditable and explainable

# When the AI is wrong, Db2 catches it before any human approves




Real run on Gemini, not staged.

The operations agent cited policy `POL-ENG-01`, claimed Deluxe was "not ≥ 90% sold", and proposed taking AC-fault room 307 **out of order**.

The manager pressed **Dry run**. Db2 executed the real stored procedure inside a transaction, recomputed sellable inventory, and **rolled back**:

> ⚠️ Taking the room out of order leaves **DLX OVERSOLD on 17–18 Nov** (the conference nights). **5 reservations** are assigned to this room.

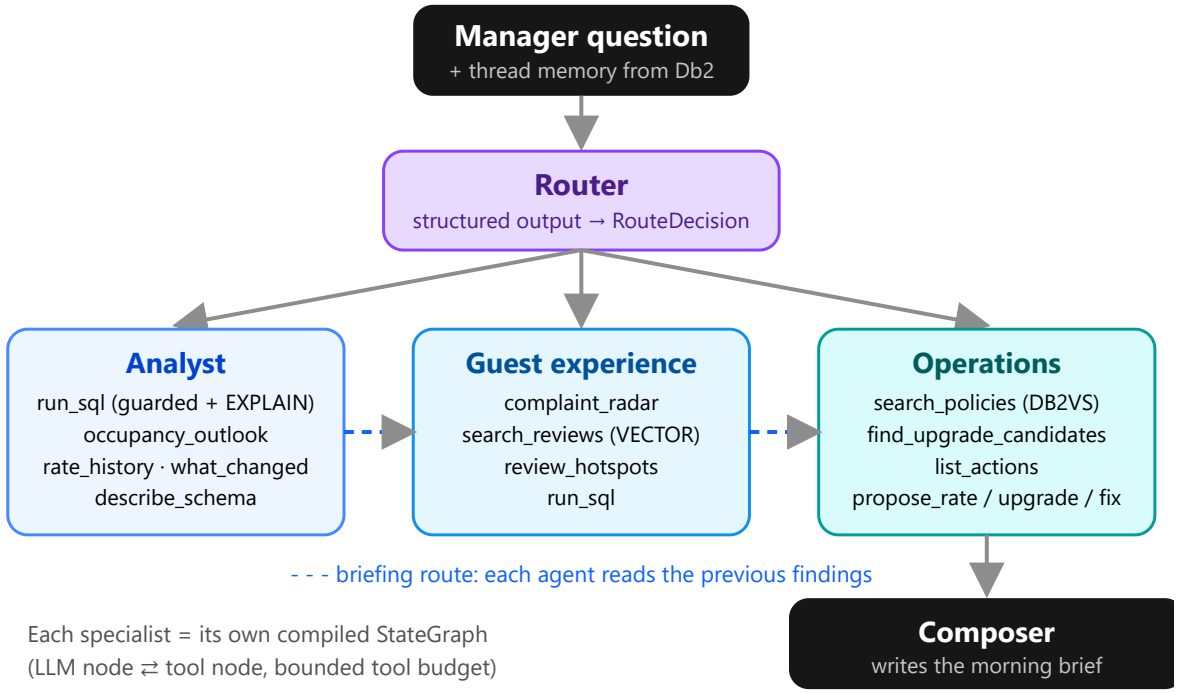
## Why this matters

- Safety comes from **Db2** (procedures, transactions, privileges, RCAC), not from hoping the LLM behaves
- The AI can be fooled or wrong; the system stays correct
- Managers see **impact before approval** and can **undo after**

SELECTED AGENTIC AI FRAMEWORK

Agentic AI framework

# LangGraph: explicit, auditable multi-agent control flow




## Why LangGraph

- **Explicit graph**: deterministic routing, bounded loops and sub-graphs. Every step is inspectable and stored in Db2.
- **Least-privilege tools per agent**: the Analyst *cannot* propose actions, and only Operations can.
- **Multi-agent collaboration**: in a briefing, each specialist builds on the previous agents' findings.

## Production-minded AI integration

- **Model**: Google Gemini 3.5 Flash through LangChain, swappable for watsonx.ai, OpenAI or Ollama
- **Quota-aware fallback chain** switches models mid-run, visible in the trace
- **Memory in Db2**: earlier turns are replayed from `AGENT_RUNS`
- **Untrusted data**: review text is never treated as instructions
- **Efficient**: batched tool calls, 3.4 LLM calls per answer on average

IBM Db2 connector

# Native `ibm_db` plus IBM's official `langchain-db2`, with two Db2 identities

### 🤖 Agent identity

`OPSAGENT`, role `HOTELOPS_AGENT`: **SELECT** plus **EXECUTE SP_PROPOSE_ACTION** only. It sees PII masked and opted-out guests hidden.

### 👔 Manager identity

Approves, rejects, dry-runs and rolls back through stored procedures. Runs the EXPLAIN cost check.

| CONNECTOR | USED FOR |
|---|---|
| `ibm_db` 3.3 (native driver) | All agent tools as LangChain `@tool`s; stored procedure calls with OUT parameters; prepared statements |
| **IBM `langchain-db2`** (`DB2VS`) | Policy handbook as a LangChain `VectorStore` on Db2 `VECTOR(384)`, through `ibm_db_dbi` |
| Local embeddings (BGE-small, ONNX) | Review and policy vectors, so text never leaves the machine to be embedded |
| Docker: Db2 Community Edition 12.1 | `docker compose up` starts Db2 and the app together |

```python
# every agent tool runs as the least-privileged identity
agent_db = get_db("agent")        # OPSAGENT, role HOTELOPS_AGENT

def run_sql(query: str) -> str:
    safe = validate_select(query)    # 1 statement, SELECT only
    check_cost(safe)                 # Db2 EXPLAIN PLAN, cost ceiling
    return agent_db.query(safe)       # Db2 privileges + RCAC apply

# agents can only PROPOSE: the procedure validates and queues
agent_db.call("HOTELOPS.SP_PROPOSE_ACTION", run_id, "RATE_CHANGE",
              json.dumps(payload), rationale, 0, OUT_TEXT)

# IBM langchain-db2: policy knowledge base on Db2 vectors
kb = DB2VS(embedding_function=LocalBgeEmbeddings(),
           table_name="POLICY_KB", client=ibm_db_dbi_conn,
           distance_strategy=DistanceStrategy.COSINE)
kb.similarity_search_with_score("upgrade a platinum guest")
```

The connection string sets `CURRENTSCHEMA=HOTELOPS`. Every credential comes from `.env`; none is committed.

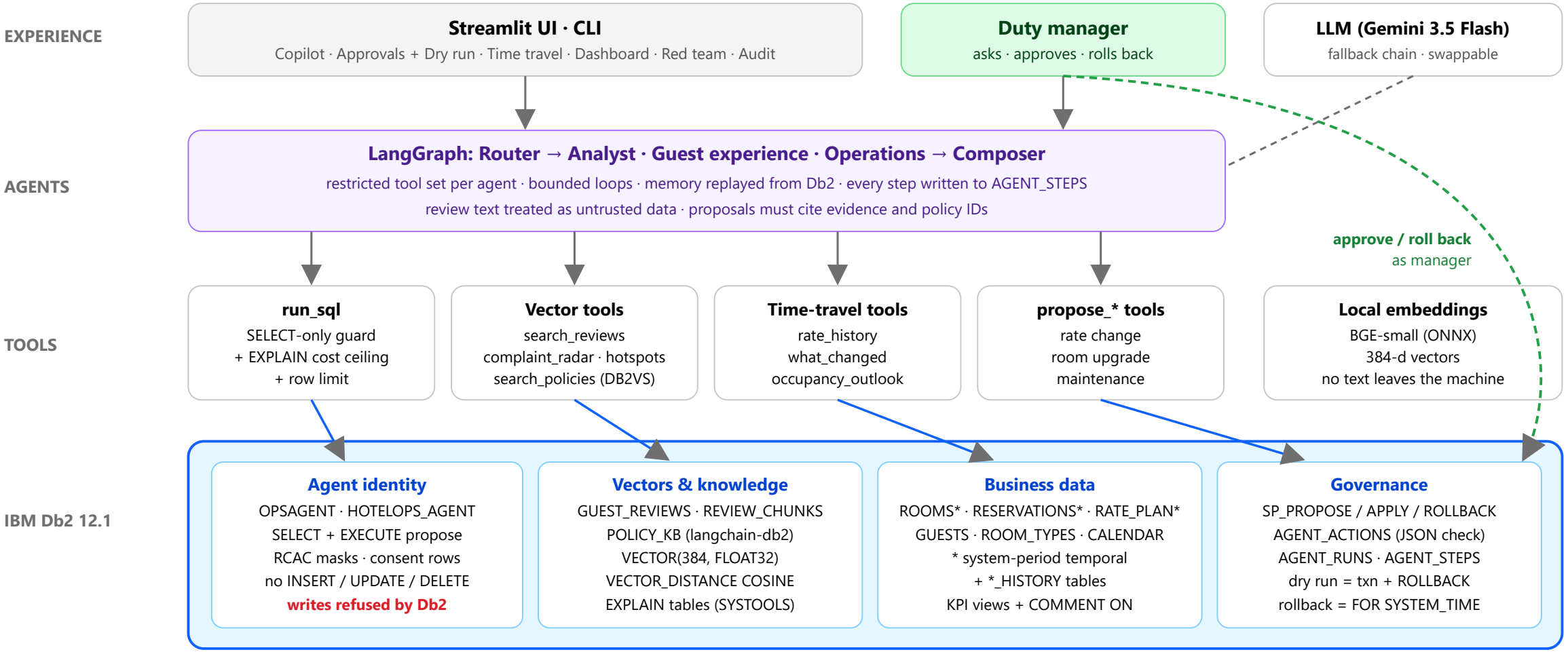
Intended Db2 integration

# Db2 is the system of record *and* the AI's control plane

| DB2 CAPABILITY | WHERE | WHAT IT DOES FOR THE COPILOT |
|---|---|---|
| VECTOR(384, FLOAT32)<br>VECTOR_DISTANCE | `REVIEW_CHUNKS`, `POLICY_KB` | Sentence-level semantic review search; a complaint radar in one query (anchor vectors `CROSS JOIN`, `ROW_NUMBER() OVER`, `GROUPING SETS`) |
| System-period + bitemporal | `RATE_PLAN`, `RESERVATIONS`, `ROOMS`, `PROMOTIONS` | "Who changed the rate?"; a trigger-free change feed; one-click rollback (`FOR SYSTEM_TIME AS OF`); `BUSINESS_TIME` promotions answer "what was a guest quoted before the correction?" |
| SQL PL procedures | `SP_PROPOSE / APPLY / REJECT / ROLLBACK` | Price band, 35% cap, no past dates, real upgrades only, free room for the whole stay, consent, duplicates, and rollback integrity |
| Transactional dry run | dedicated unit of work + `ROLLBACK` | Runs the real apply procedure, diffs the impact (oversold nights, guests to move, revenue at stake), then persists nothing |
| RCAC masks + row permission | `GUESTS` | E-mail and phone masked for the AI; guests who withdrew consent are invisible to it (DPDP Act 2023) |
| Roles & GRANT / REVOKE | `HOTELOPS_AGENT` | The AI can read and propose, never write. `CREATETAB` and `IMPLICIT_SCHEMA` are revoked from PUBLIC |
| EXPLAIN PLAN | `SYSTOOLS.EXPLAIN_*` | The optimizer costs every AI-written query before it runs; runaway plans are refused (legitimate queries cost 78–2,089 timerons) |
| JSON_EXISTS / JSON_VALUE | `AGENT_ACTIONS` | Validated structured proposals (check constraint) parsed inside the procedures |
| Views, MQT, COMMENT ON, SYSCAT | `V_DAILY_KPIS`, `MQT_DAILY_KPIS` | A semantic layer for text-to-SQL; exact column lists are fed to the agents, so they don't guess |
| Tamper-evident audit | `AGENT_STEPS` trigger + `HASH` | Every tool call SHA-256 hash-chained by Db2 and anchored outside it: an edited step, or even a fully rebuilt ledger, is caught; replay without an LLM |

System architecture

# Four layers; the guarantees live in the bottom one



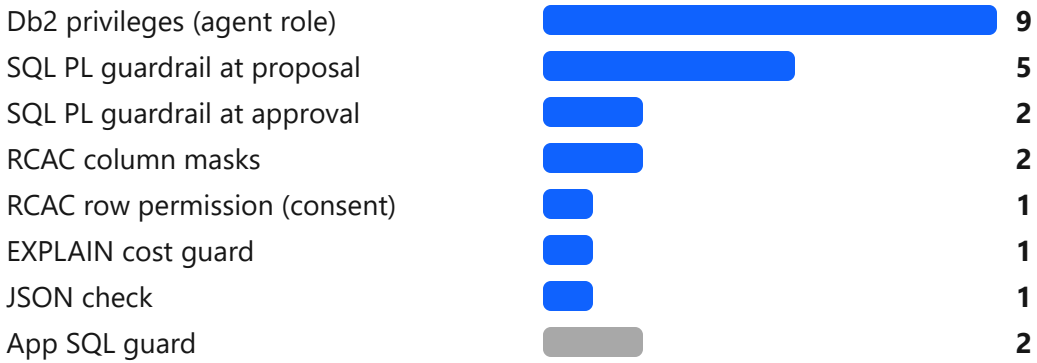
Workflow

# Seven steps, five Db2 checkpoints

| # | STEP | WHAT HAPPENS |
|---|---|---|
| 1 | **Question** | The manager asks; the run is logged in `AGENT_RUNS`, with memory replayed from earlier turns of the thread |
| 2 | **Routing** | The LLM router returns a typed `RouteDecision`, with a keyword fallback if structured output fails |
| 3 | **Evidence** Db2 | Specialists query as `OPSAGENT`: SQL costed by EXPLAIN, vector search, time travel. Privileges and RCAC apply |
| 4 | **Proposal** Db2 | `SP_PROPOSE_ACTION` validates the JSON, references, price band, dates, consent and duplicates, then queues it as PENDING |
| 5 | **Dry run** Db2 | The real apply procedure runs in a transaction; the UI shows the diff and warnings, then `ROLLBACK` |
| 6 | **Approval** Db2 | A human approves, or Db2 auto-applies the action if it is within the earned-autonomy limits ( `AUTONOMY_POLICY` ); `SP_APPLY_ACTION` re-checks the 35% cap, upgrade tier and room availability, then applies the change to the temporal tables |
| 7 | **Undo** Db2 | `SP_ROLLBACK_ACTION` restores the rows `AS OF` the moment before the change, and refuses if someone changed them again |

## Red-team scoreboard: 24/24 attacks blocked

The attacks bypass the app and use the agent's own Db2 credentials. Each runs in a transaction that is rolled back.




Attacks include deleting guests, a ₹500 suite, self-approval, dumping PII, bypassing consent, a cross-join overload, a −40% price swing, a fake "upgrade", and **a prompt injection hidden in a guest review**.

Feasibility: built and measured

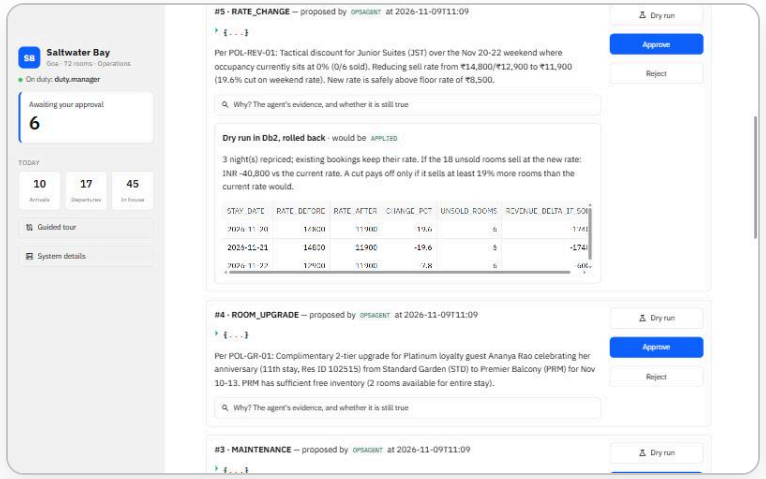
# Measured results, not just intentions

**30/30**
accuracy evaluation: KPIs, time travel, vector search, change feed and safety, with ground truth computed in Db2

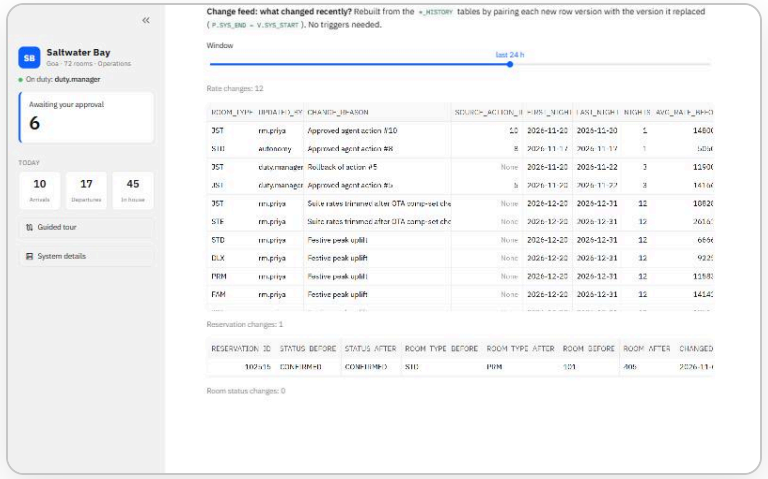
**24/24**
red-team attacks blocked; the database is proven unchanged afterwards

**10,000**
random proposals fuzzed into the Db2 guardrails: every decision matches an independent oracle

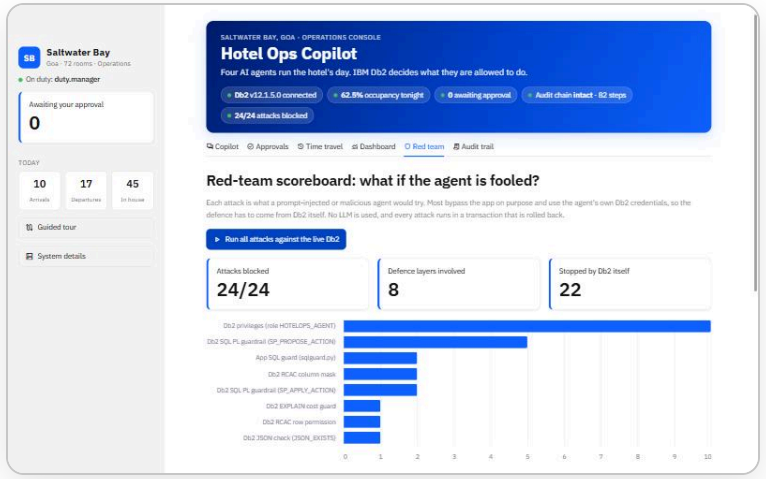
**26/26**
live Db2 checks of every claim ( `verify.py` / Proofs tab); 106 tests, 87% coverage




**Dry run** shows the break-even: the agent's Junior Suite cut pays off only if it sells **19% more rooms**.




**Change feed** from history tables: before and after, who, why, and the action ID.




**Red team**: one click runs 24 attacks against the live Db2.

# Better decisions every morning, without giving up control

### 💰 Revenue

- Soft nights are found days earlier and discounted **within policy**
- Sold-out conference nights are priced up, not left on the table
- The dry run shows money at stake and break-even before approval

### ⭐ Guest experience

- Faulty rooms are found from the **first few** reviews, not the fiftieth
- VIPs and special occasions are recognised automatically
- Process issues (Wi-Fi, check-in) are separated from room faults

### 🛡️ Trust & compliance

- The AI never writes data; every action is approved, audited and reversible
- PII never reaches the model; consent is enforced by Db2 (DPDP Act)
- Resistant to prompt injection by design

### Use case: Monday, 07:30, Saltwater Bay (Goa)

The duty manager clicks **Morning brief**. Within minutes the copilot reports tonight's occupancy, flags AC faults in rooms 307/312 and no hot water in 512, spots zero suite pick-up for 20–23 Nov and the sold-out conference nights, and queues 6 actions: two repairs, an anniversary upgrade for a Platinum guest, and three rate changes. Each action can be dry-run first (in one recorded run this caught an action that would oversell Deluxe). The manager approves, and can undo any of them later.

### Who can use it and how it scales

- **Independent and mid-size hotels** that lack revenue-management tools
- **Hotel chains**: the same Db2 row permissions can scope each manager and agent to their own property
- **Any regulated domain** where AI must propose and humans must approve: hospitals, retail, banking ops
- The domain is swappable; the **Db2 governance pattern is reusable**

IMPLEMENTATION PLAN, TECH STACK AND REPRODUCIBILITY

# Reproducible in one command, built to be judged

### Tech stack

- **Database**: IBM Db2 12.1 Community Edition (Docker)
- **Connectors**: `ibm_db`, IBM `langchain-db2`, `ibm_db_dbi`
- **Agents**: LangGraph + LangChain
- **LLM**: Gemini 3.5 Flash (fallback chain); watsonx.ai, OpenAI and Ollama ready
- **Embeddings**: BGE-small (fastembed, ONNX)
- **UI**: Streamlit + CLI; **tests**: pytest

### Run it

```
# Db2 + app, no local Python
cp .env.example .env
docker compose up -d
# open http://localhost:8501

# or step by step
bash scripts/quickstart.sh
pytest -q tests/           # 106 tests
python src/redteam.py      # 24/24
python src/evaluate.py     # 30/30
```

It works without an API key too: approvals, dry run, time travel, radar, red team and replay of recorded runs.

### Plan and roadmap

- ✅ Schema, temporal tables, vectors, SQL PL guardrails, RCAC
- ✅ LangGraph agents, tools, memory, fallback
- ✅ Dry run, radar, change feed, red team, evaluation, earned autonomy
- ✅ Docker, docs, screenshots
- → Next: PMS and channel-manager connectors, forecasting with Db2 ML / watsonx.ai, Granite as the default model, multi-property row permissions

**Hotel Ops Copilot**

AI that proposes. Db2 that decides, dry-runs and can undo.

Clark Fernandes · solo · GitHub **clarkf19**

IBM Db2 12.1 · LangGraph · Gemini